

PROJECT REPORT

(Project Term January – May 2026)

Distributed File Upload System with DevOps Pipeline

(Docker | Maven | Kubernetes | GitHub Actions CI/CD)

Submitted by

Punit Kumar

Registration Number : 12307813

Course Code : INT332

Under the Guidance of

Ms. Jatinder Kaur



School of Computer Science and Engineering

Lovely Professional University, Phagwara, Punjab

DECLARATION

I hereby declare that the project work entitled "**Distributed File Upload System with DevOps Pipeline**" is an authentic record of my own work carried out as requirements of the Project for the award of B.Tech Degree in Computer Science and Engineering from Lovely Professional University, Phagwara, under the guidance of **Ms. Jatinder Kaur**, during May 2026. All the information furnished in this project report is based on my own intensive work and is genuine.

Name of Student : Punit Kumar

Registration No. : **12307813**

Date: 22-05-2026

Signature of Student

CERTIFICATE

This is to certify that the declaration statement made by the student is correct to the best of my knowledge and belief. He has completed this Project under my guidance and supervision. The present work is the result of his original investigation, effort and study. No part of the work has ever been submitted for any other degree at any University.

The Project is fit for the submission and partial fulfillment of the conditions for the award of B.Tech Degree in B.Tech Degree in Computer Science and Engineering from Lovely Professional University, Phagwara.

Signature and Name of the Mentor

School of Computer Science and Engineering,
Lovely Professional University, Phagwara, Punjab.

Date : 22-05-2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty mentor for providing continuous guidance, motivation, and support throughout the development of this project. Their expertise and constructive feedback were invaluable in shaping the direction and quality of this work.

I am also grateful to the School of Computer Science and Engineering, Lovely Professional University, for providing the necessary infrastructure, resources, and a conducive environment to carry out this project.

Special thanks to the open-source communities behind Spring Boot, Docker, Kubernetes, and GitHub Actions whose well-documented tools made this project possible.

Finally, I thank my family and friends for their constant encouragement and moral support throughout this endeavour.

Punit Kumar
12307813

TABLE OF CONTENTS

Section	Title	Page
Declaration	Student Declaration	(ii)
Certificate	Faculty Certificate	(iii)
Acknowledgement	Acknowledgement	(iv)
1	Introduction	1
2	Scope of the Study / Problem Statement	3
3	Existing System	4
4	Problem Analysis	5
5	Software Requirement Analysis	6
6	Design	8
7	Testing	10
8	Implementation	11
9	System Snapshots	12
10	Bibliography	14

1. INTRODUCTION

In today's digital world, the need to handle large files reliably over a network is a core requirement of modern web applications. Traditional file upload approaches that send a file in a single HTTP request are fragile — a network interruption causes the entire transfer to fail, large files exhaust server memory, and a single server deployment cannot scale to handle high traffic. These shortcomings make traditional upload systems unsuitable for enterprise use.

This project, the Distributed File Upload System with DevOps Pipeline, solves these problems through two complementary designs: a chunked file transfer mechanism on the application side, and a fully automated DevOps pipeline on the operations side. Files of any type and size are split into small chunks by the browser, transferred independently, reassembled by the Spring Boot backend, and automatically backed up to a private GitHub repository — regardless of file size or network conditions.

On the DevOps side, every code change triggers a GitHub Actions pipeline that automatically runs unit tests, packages the application with Maven, builds a Docker image, pushes it to Docker Hub, and deploys it to a Kubernetes cluster — achieving continuous integration and continuous delivery (CI/CD) with zero manual steps.

1.1. PROJECT BACKGROUND

The project was conceived as a practical solution to a real engineering challenge: how to reliably upload files larger than 100 MB to a server running in a containerised, replicated environment. GitHub's file API limits individual files to 25 MB — which made direct storage impossible. The solution was to split merged files into 20 MB parts and push each part using the GitHub Contents API, with a manifest file tracking how to reassemble them.

The DevOps infrastructure was designed in parallel — ensuring that as the application code evolved, each new version could be tested, containerised, and deployed automatically without developer intervention.

1.2. PROJECT OBJECTIVES

- Implement chunked file upload in a Spring Boot REST API to handle files of arbitrary size.
- Automatically merge received chunks, validate integrity, and push file parts to a private GitHub repository.
- Build a CI/CD pipeline: GitHub Actions triggers Maven tests, Docker build/push, and Kubernetes deployment on every code commit.
- Deploy the application on Kubernetes (Minikube) with auto-scaling (2–6 pods) and rolling updates (zero downtime).
- Build automated validation and reconstruction workflows in the storage repository to verify all parts are received correctly and to reconstruct original files as downloadable artifacts.

2. SCOPE OF THE STUDY (PROBLEM STATEMENT)

2.1. PROBLEM STATEMENT

Traditional file upload systems suffer from several critical limitations that make them unsuitable for production environments:

- Single-request failures: One network drop fails the entire upload with no resume capability.
- Memory exhaustion: Loading a 500 MB file into server RAM crashes the application.
- No redundancy: Files exist only on one server; hardware failure destroys the data permanently.
- No integrity checks: Corrupted or incomplete uploads are silently accepted.
- Manual deployment: Developer must SSH into a server and run commands manually for every release.
- No auto-scaling: A single server cannot handle sudden spikes in upload traffic.

Additionally, GitHub's Content API enforces a 25 MB per-file limit — making direct backup of large files impossible without a split-and-store strategy.

2.2. SCOPE OF SOLUTION

This project addresses all of the above by implementing:

- Chunked transfer at the browser layer — only individual chunks need to be re-sent on failure.
- Stream-based merging in Spring Boot — no large file is ever loaded fully into RAM.
- Automated GitHub backup — every merged file is split into 20 MB parts and pushed to a private repo.
- Automated validation workflow — GitHub Actions checks all parts are present before reconstruction.
- Fully automated CI/CD — no manual steps between a code commit and a live deployment.
- Kubernetes HPA — pods scale automatically from 2 to 6 based on CPU load.

3. SOFTWARE REQUIREMENT ANALYSIS

5.1. INTRODUCTION

Software requirements define the functions the system must perform, the constraints it must operate under, and the quality attributes it must meet. They are divided into Functional Requirements (what the system does) and Non-Functional Requirements (how well it does it).

5.2. GENERAL DESCRIPTION

The Distributed File Upload System is a web application that allows users to upload files of any size and type through a browser. The backend is a Spring Boot application that receives file chunks, assembles the original file, validates it, and backs it up to GitHub. The system is hosted on a Kubernetes cluster and delivered through a CI/CD pipeline.

5.3. SPECIFIC REQUIREMENTS

5.3.1. Functional Requirements

- FR1 — Chunked Upload: The system shall accept files split into numbered chunks via HTTP POST to /api/upload.
- FR2 — Chunk Storage: Each chunk shall be stored in a temporary directory indexed by fileId and chunkIndex.
- FR3 — Merge: When all expected chunks for a fileId are received, the system shall concatenate them in order and produce the original file in uploads/final/.
- FR4 — Validation: The merged file byte count shall be validated against the expected total.
- FR5 — GitHub Backup: The merged file shall be split into ≤ 20 MB parts and each part pushed to a private GitHub repository using the Contents API.
- FR6 — Manifest: A text manifest file listing all parts, sizes, and reassembly instructions shall be committed alongside the parts.
- FR7 — Validation Workflow: A GitHub Actions workflow shall verify all parts are present and sizes are consistent after every push to the storage repository.
- FR8 — Reconstruction: A GitHub Actions workflow shall reconstruct the original file from its parts and make it available as a downloadable artifact.
- FR9 — CI/CD: Every push to the main branch of the code repository shall trigger an automated pipeline: test → package → docker build → docker push → kubectl deploy.
- FR10 — Auto-Scaling: The Kubernetes HPA shall scale pods between 2 and 6 based on CPU utilisation.

5.3.2. Non-Functional Requirements

Category	Requirement
Performance	The system shall handle files up to 1 GB; each 20 MB part shall upload in under 30 seconds on a 10 Mbps connection.

Category	Requirement
Reliability	File reconstruction shall be byte-perfect (SHA-256 verified). No data loss shall occur if a chunk upload fails.
Scalability	Kubernetes HPA shall respond to load increases within 60 seconds by adding pods.
Security	All credentials (PAT, Docker token) shall be stored as GitHub Secrets or Kubernetes Secrets — never hardcoded.
Maintainability	All infrastructure-as-code (Kubernetes YAML, Dockerfile, workflows) shall be version-controlled in Git.
Portability	The Docker image shall run identically on Windows, macOS, and Linux.
Availability	Rolling update strategy shall ensure zero downtime during deployments (maxUnavailable: 0).

5.4. TECHNOLOGY STACK

Technology	Version	Role
Spring Boot	3.x	REST API, chunk management, file merge logic
Java	17 LTS	Primary programming language
Apache Maven	3.8+	Build tool, dependency management, test runner
Docker	24+	Application containerisation and image creation
Docker Hub	Cloud	Docker image registry (punitkumar4871/...)
Kubernetes	1.28+	Container orchestration, scaling, deployment
Minikube	1.32+	Local Kubernetes cluster for development
GitHub Actions	Cloud	CI/CD pipeline automation
GitHub Contents API	v3	Direct REST API for pushing file parts to GitHub
Python	3.10+	Validation and merge scripts in storage repo

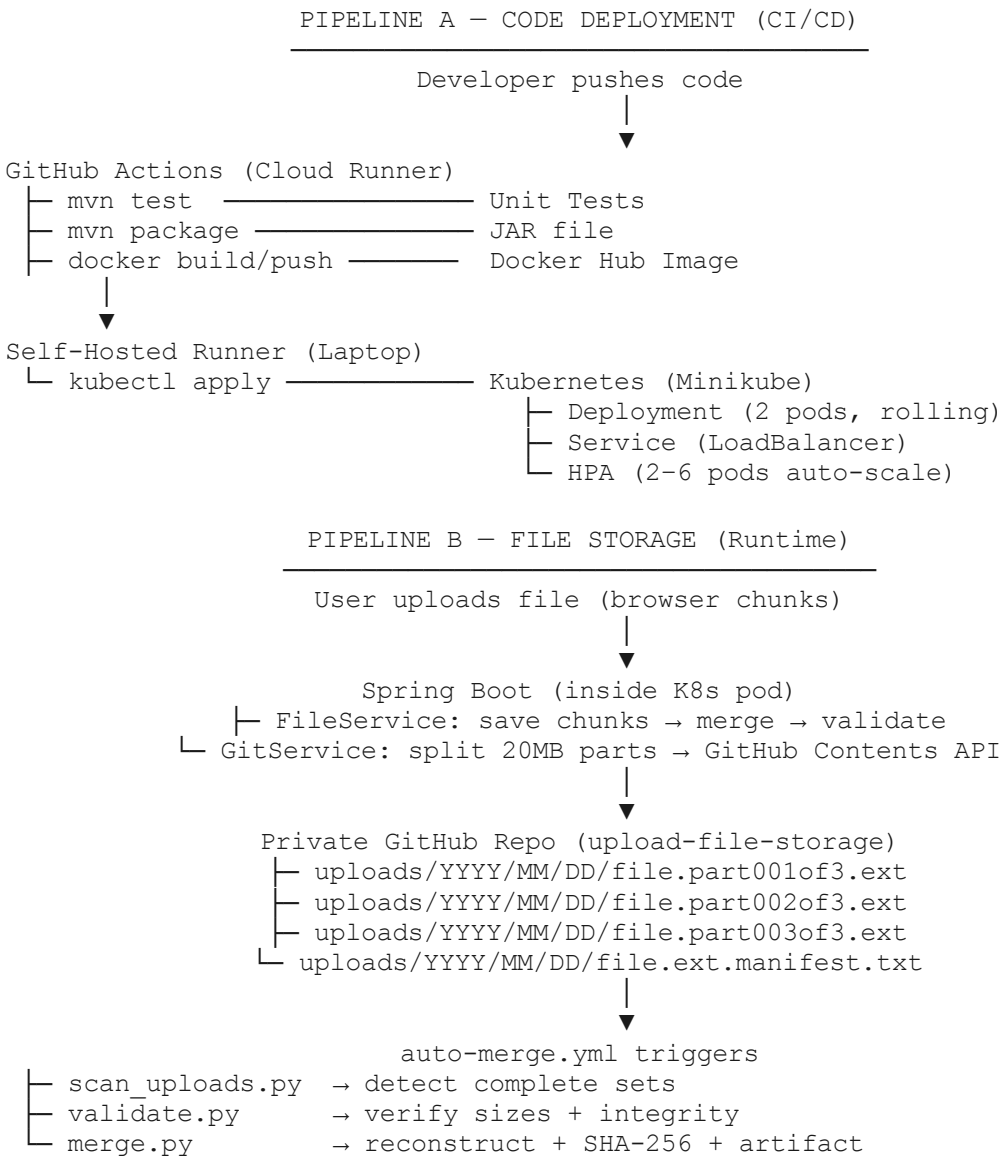
4. DESIGN

6.1. SYSTEM DESIGN

The system is composed of two independent, cooperating pipelines:

- Pipeline A (CI/CD) — Triggered by code commits; handles build, containerisation, and deployment.
- Pipeline B (File Storage) — Triggered at runtime by file uploads; handles backup to GitHub.

Figure 6.1 — Full System Architecture



6.2. DESIGN NOTATIONS

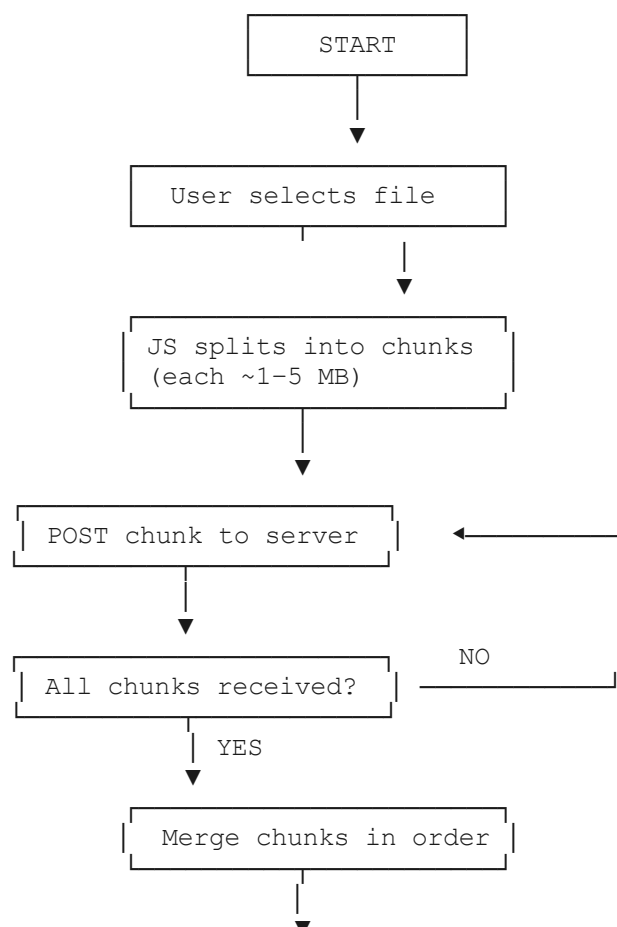
6.2.1. Module Breakdown

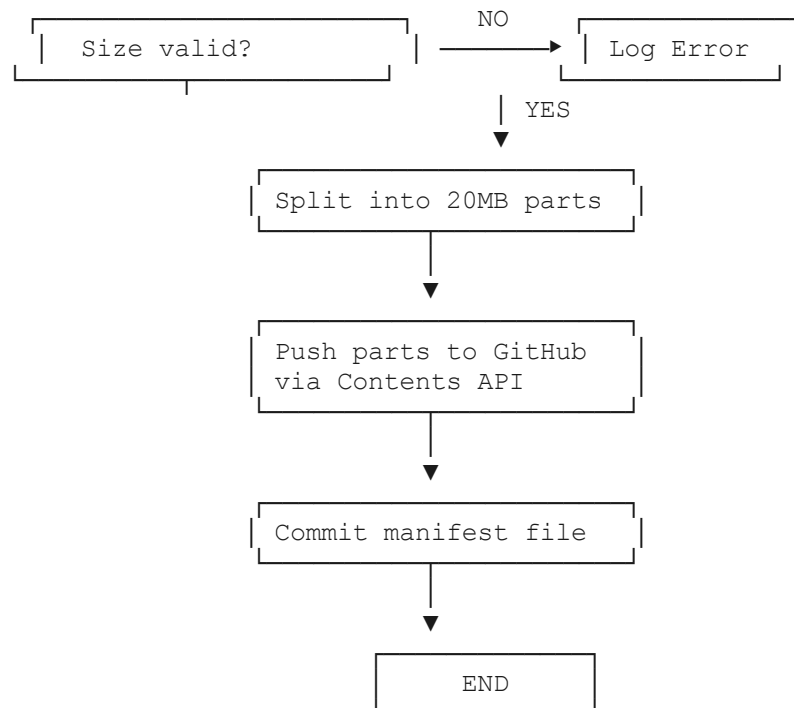
Module	File	Responsibility
Upload Controller	ChunkUploadController.java	REST endpoint; receives chunks; calls FileService
File Service	FileService.java	Saves chunks; detects completion; merges; validates
Git Service	GitService.java	Splits merged file; pushes parts via GitHub API
Scan Script	scan_uploads.py	Finds complete upload sets; skips already-merged
Validate Script	validate.py	Checks all parts present; verifies sizes
Merge Script	merge.py	Reconstructs file; computes SHA-256; creates artifact
CI/CD Workflow	ci-cd.yml	Build → Test → Docker → K8s deploy on code push
Auto-Merge Workflow	auto-merge.yml	Validate + merge on push to uploads/**

6.4. FLOWCHARTS

6.4.1. File Upload Flowchart

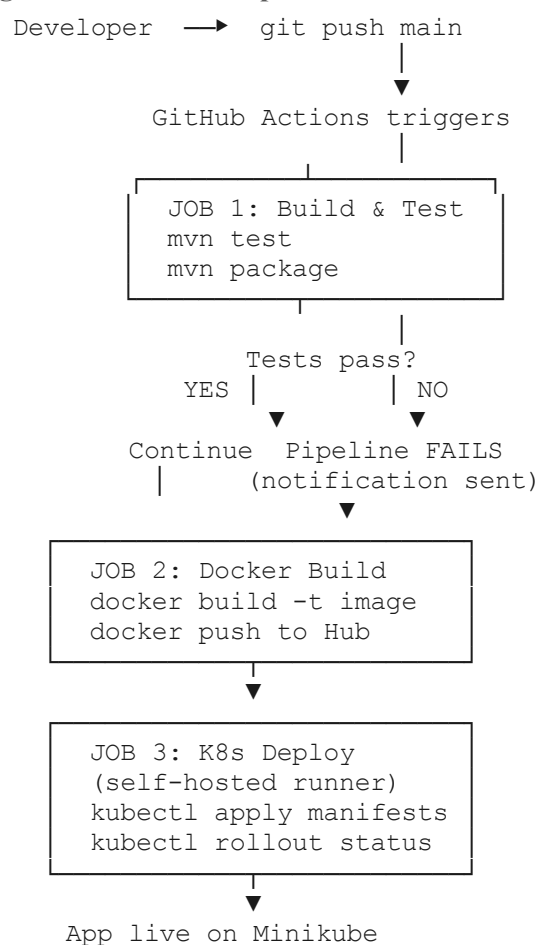
Figure 6.4 — File Upload and Backup Flowchart





6.4.2. CI/CD Pipeline Flowchart

Figure 6.5 — CI/CD Pipeline Flowchart



5. TESTING

7.1. TESTING THE PROJECT

7.1.1. Test Results — File Upload

File	Size	Chunks (browser)	GitHub Parts	Reconstructi on	SHA-2 56 Match
MSB (1).exe	231.6 MB	232	12 × 20 MB	Successful	Yes
50mb.pdf	52.1 MB	~53	3 parts	Successful	Yes
notes.docx	2.3 MB	3	1 part	Successful	Yes

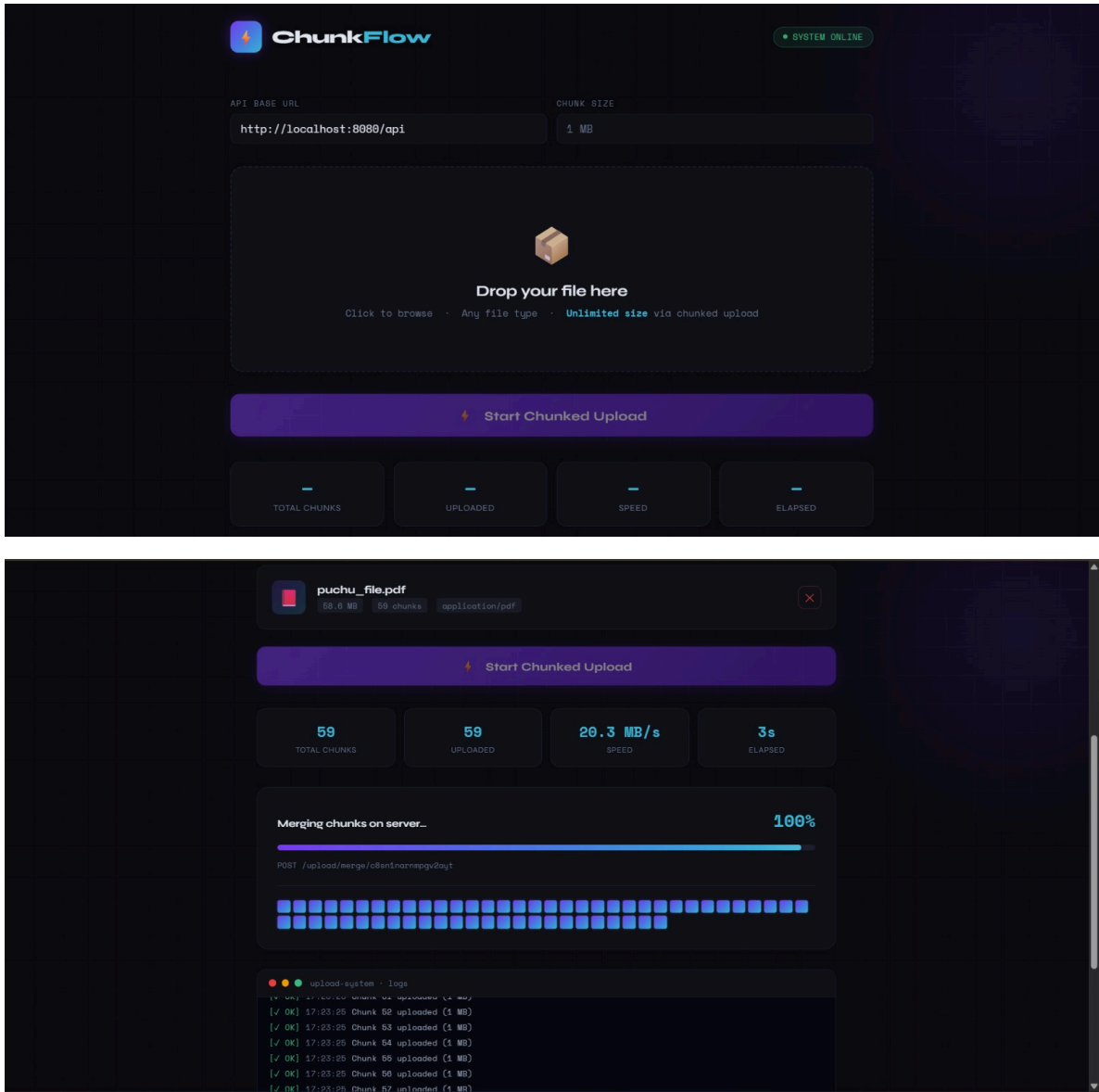
7.1.2. Test Results — CI/CD Pipeline

Job	Status	Duration	Notes
Build & Test (mvn test + package)	PASSED	~45 sec	All JUnit tests pass
Docker Build & Push	PASSED	~1 min	Image pushed to Docker Hub
Deploy to Kubernetes (self-hosted)	PASSED	~30 sec	Pods rolling updated, 2/2 Running

6. SYSTEM SNAPSHOTS AND SOURCE CODE

The following snapshots describe the key screens and log outputs observed during the project. Actual screenshots should be inserted at the indicated positions.

10.1. FILE UPLOAD UI



The browser-side upload interface shows a file picker and a progress bar. JavaScript splits the selected file into chunks and sends each independently via POST requests.

10.2. SPRING BOOT TERMINAL LOGS

Figure 10.1 — Spring Boot Terminal Log During EXE Upload

```
[FileService] Saved chunk 229 for fileId=662x6fqx4mpf2kgr0
[FileService] Saved chunk 230 for fileId=662x6fqx4mpf2kgr0
[FileService] Saved chunk 231 for fileId=662x6fqx4mpf2kgr0
[FileService] Merged 232 chunks -> uploads\final\MSB (1).exe
[FileService] Deleted temp dir for fileId=662x6fqx4mpf2kgr0
```

```

[GitService] ===== GITHUB PUSH =====
[GitService] File      : MSB (1).exe
[GitService] Token set  : YES (ghp_eTtb...)
[GitService] Repo      : punitkumar4871/upload-file-storage
[GitService] File size  : 231.6 MB
[GitService] Splitting into 12 x 20MB parts...
[GitService] Part 1/12 (20.0 MB) → committed OK
[GitService] Part 2/12 (20.0 MB) → committed OK
... (parts 3-11 committed OK)
[GitService] Part 12/12 (11.6 MB) → committed OK
[GitService] Manifest committed: uploads/2026/05/21/MSB_1.exe.manifest.txt
[GitService] Result: 12/12 parts committed.

```

10.3. GITHUB ACTIONS — AUTO-MERGE WORKFLOW

Figure 10.2 — GitHub Actions Auto-Merge Workflow Output

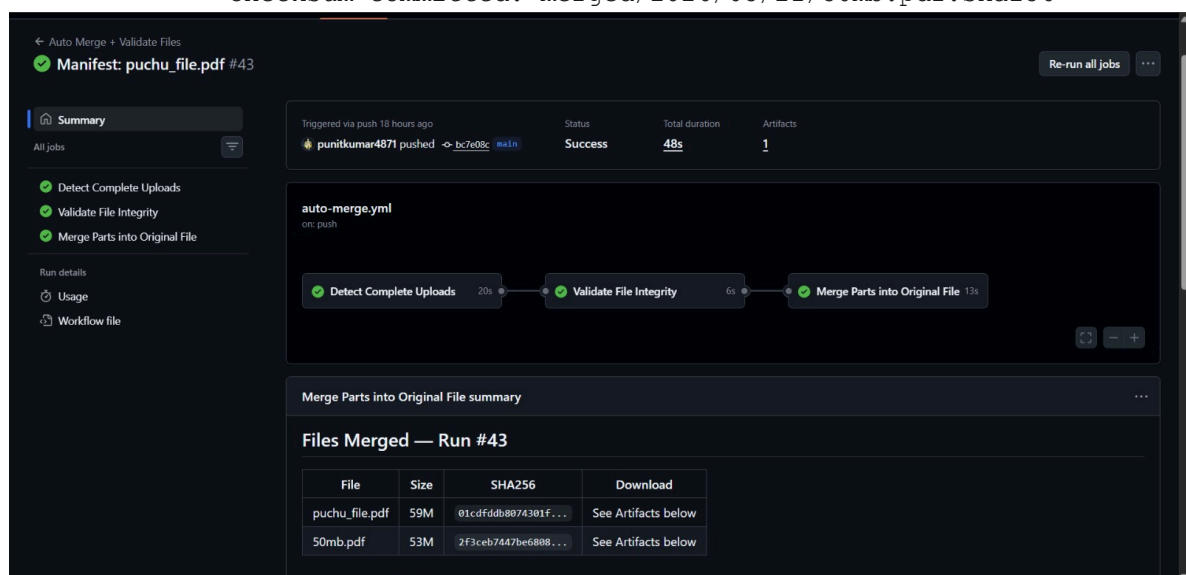
```

✅ JOB 1: DETECT
[SCAN] MSB (1).exe - ALREADY MERGED - skipping
[SCAN] 50mb.pdf - Parts found: 3/3 - COMPLETE
Files ready to merge: 1

✅ JOB 2: VALIDATE
Part 1/3: 50mb.part001of3.pdf (20.0 MB)
Part 2/3: 50mb.part002of3.pdf (20.0 MB)
Part 3/3: 50mb.part003of3.pdf (12.1 MB)
RESULT: PASSED - all checks OK

✅ JOB 3: MERGE
Merging 3 parts → 50mb.pdf
SHA-256: a3f8e2...9b1c
Artifact uploaded: merged-files-47.zip
Checksum committed: merged/2026/05/21/50mb.pdf.sha256

```



10.4. SOURCE CODE

<https://github.com/punitkumar4871/distributed-file-upload-system>

<https://github.com/punitkumar4871/upload-file-storage>

12. BIBLIOGRAPHY

Ref.	Source	URL / Details
[1]	Spring Boot Official Documentation	https://docs.spring.io/spring-boot/docs/current/reference/html/
[2]	GitHub REST API — Repository Contents	https://docs.github.com/en/rest/repos/contents
[3]	Docker Documentation	https://docs.docker.com/
[4]	Kubernetes Documentation	https://kubernetes.io/docs/home/
[5]	Minikube Getting Started	https://minikube.sigs.k8s.io/docs/start/
[6]	GitHub Actions — Workflow Syntax	https://docs.github.com/en/actions/using-workflows/workflow-syntax-for-github-actions
[7]	Apache Maven Documentation	https://maven.apache.org/guides/index.html
[8]	Docker Hub — punitkumar4871	https://hub.docker.com/u/punitkumar4871
[9]	Java HttpClient — JDK 11+	https://docs.oracle.com/en/java/javase/17/docs/api/java.net.http/java/net/http/HttpClient.html
[10]	Python hashlib — SHA-256	https://docs.python.org/3/library/hashlib.html
[11]	Kubernetes HPA Documentation	https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/
[12]	Spring Multipart File Upload	https://spring.io/guides/gs/uploading-files/

All URLs were last accessed on 22 May 2026

